

Claude Code创始人分享了他最新的15条CC使用技巧!

Original 花叔 花叔 2026年3月30日 00:44

Boris Cherny又出来发长帖了。



上次他分享13条Claude Code使用技巧是1月份的事，那篇我写完之后反响不错。这次他又搞了15条，但内容和上次完全不一样。像是之前说的，从Claude团队人员做的新功能的方向和他们的发言来看，他们是践行eat your own shit理念的，Boris不止是Claude Code的开发者，也是自己产品最重要的用户之一。

我翻完整个thread的第一反应是：Claude Code这几个月的进化速度有点吓人。

很多功能我之前没想到他们竟然做出来了：手机写代码、Telegram推消息给正在跑的会话、/loop自动循环巡逻、/batch同时开上千个agent做代码迁移。有些东西明显带着OpenClaw的影子，OpenClaw先做了，CC看到了可行性，然后用更成熟的工程能力实现了。

不过说实话，我个人每天用Claude Code的时长是远高于OpenClaw的。毕竟CC背后是一个成熟团队的产品，稳定性更好，在实际的编程和写作场景下表现都更稳。OpenClaw的优势在探索性和灵活性，但日常干活我还是更依赖CC。

下面逐条聊。

1. Claude Code有移动端了

Boris说他很多代码是在iOS App上写的。

这个功能在Claude App里，左侧切到Code标签页就能用。iOS端去年10月就集成了，Android今年初跟进。手机上没法做复杂的编码工作，但用来看看进度、给一个跑了半小时的会话追加指令、或者在通勤时让Claude帮你处理个简单任务，体验还不错。

不过要注意，手机端目前不能凭空启动一个全新的Code会话，它更多是和下一条要讲的Remote Control配合使用，控制你本地已经在跑的会话。

CC显然看到了需求：程序员不是24小时坐在电脑前的，移动场景下的AI编程交互是迟早要做的事（好嘛...程序员更没有所谓的下班时间了...你们任何时候都可以指挥AI工作...以及，你们的老板可以指挥你指挥AI工作...

下载 Claude iOS/Android App → 左侧 Code 标签页：
<https://apps.apple.com/app/claude-by-anthropic/id6473753684>

2. 跨设备无缝切换会话

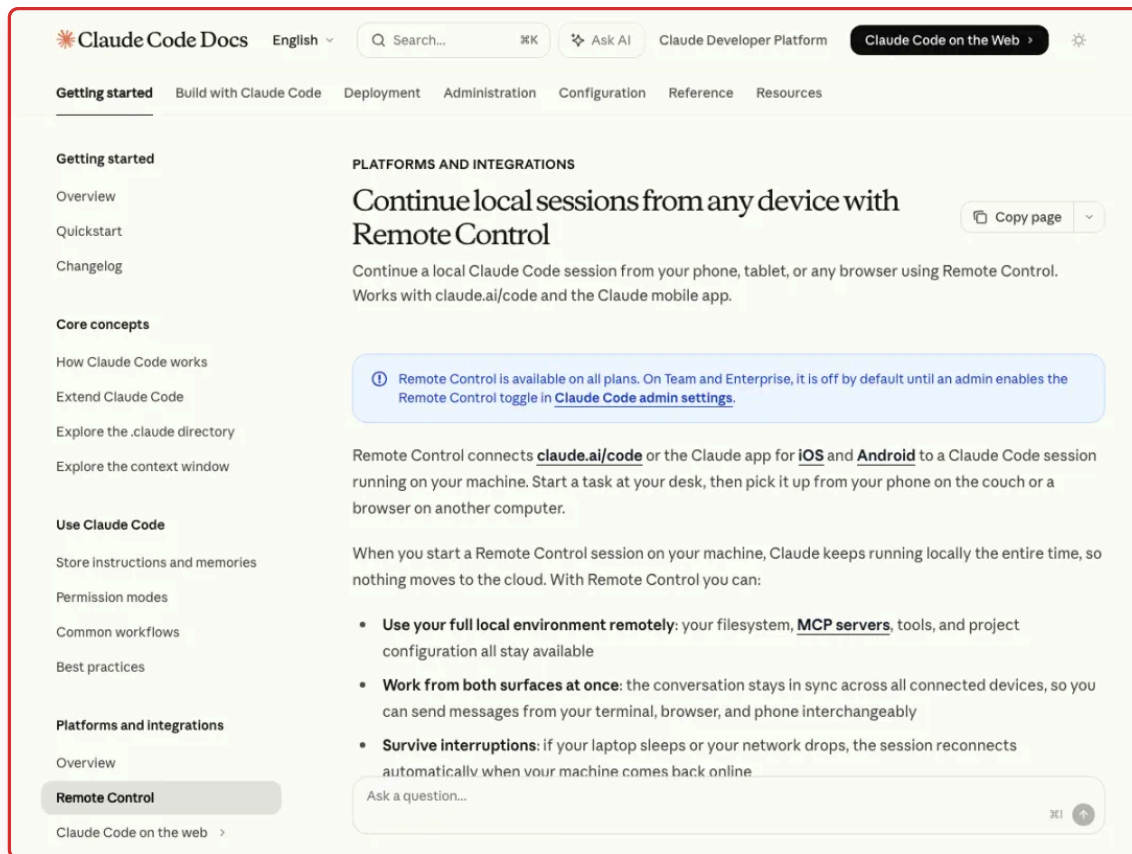
两个命令，解决两个不同问题：

Remote Control (`claude --remote-control`或会话内/rc)：在终端生成一个QR码，手机扫码后就能在claude.ai或Claude App上实时控制这个会话。注意，会话始终在你的本地机器运行，手机只是远程控制面板。2月25日正式发布，v2.1.58版本。

Teleport (`claude --teleport`或/teleport)：把Web端的云端会话拉到本地终端继续。它会自动验证仓库状态、切换到远程会话的分支、加载完整对话历史。这是单向的，只能从Web拉到本地。

Boris说他直接在设置里开了「Enable Remote Control」，默认始终开启。

你可以在公司电脑上启动一个CC会话做重构，下班路上用手机Remote Control看进度、批准权限请求、追加指令。到家后在个人电脑上继续。



或者你在 `claude.ai/code` 上开了个云端任务，做到一半想在本地终端接手，--teleport 一下就行。

会话不再绑定在某台设备上，而是跟着你走。

这里有个实用场景：长任务跑到一半需要你审批权限（比如要不要执行某个危险命令），你不在电脑前怎么办？以前只能等回去。现在 Remote Control 可以把权限请求转到手机上，你在地铁里就能批。

官方文档：<https://code.claude.com/docs/en/remote-control>

3. `/loop`和`/schedule`：最强大的两个新功能

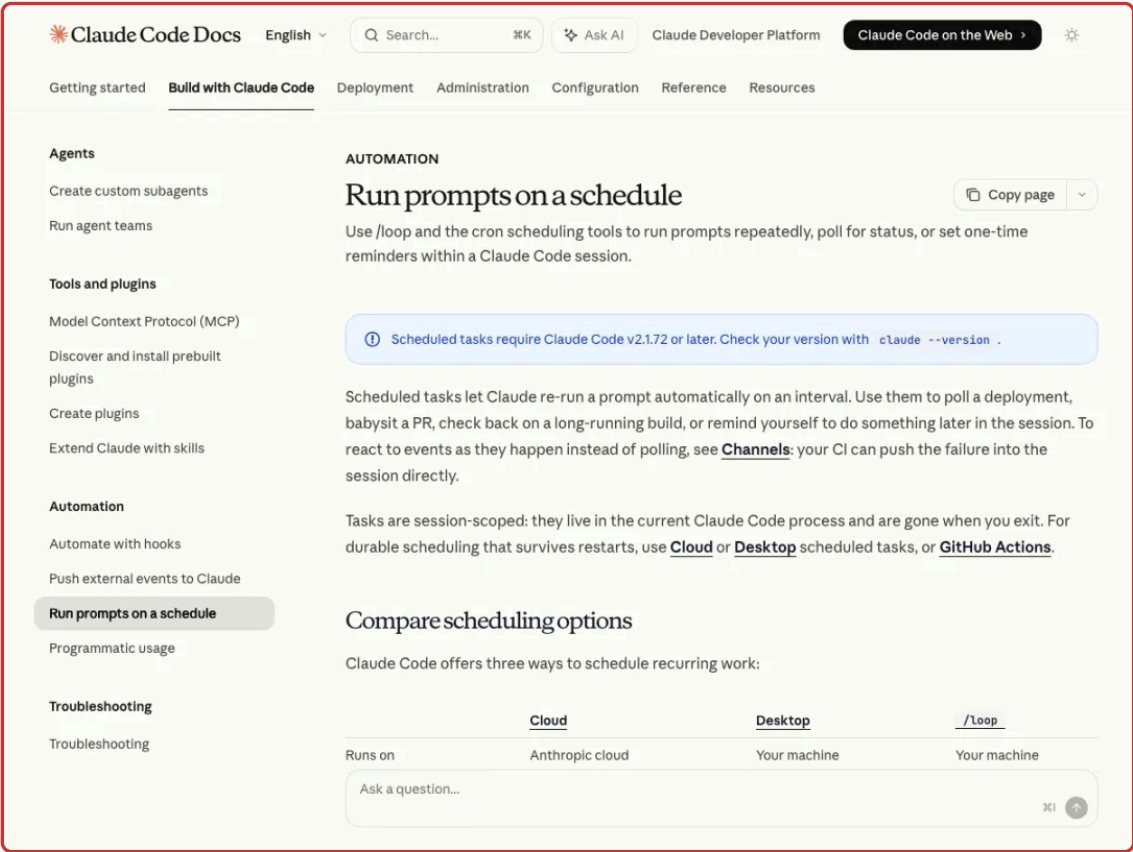
Boris说这两个功能是CC最强大的功能。他跑着一堆自动化循环：

- `/loop 5m /babysit`：每5分钟自动处理code review、自动rebase、自动回复PR评论
- `/loop 1h /tidy`：每小时自动清理代码库里需要维护的东西
- `/loop 6h /dep-upgrade`：每6小时自动升级依赖并提PR

/loop是在当前CLI会话里循环执行，退出会话就消失，创建后3天自动过期，单会话最多50个。/schedule是创建持久化定时任务，在Desktop的Cowork里管理，重启后仍然在跑。两个都是3月7日发布的，v2.1.71版本。

怎么选？短期监控用/loop（盯一小时部署），长期自动化用/schedule（每天早上跑测试）。

这个功能的灵感明显来自OpenClaw的自动化巡逻机制。



OpenClaw很早就有类似的HeartBeat概念，让AI按固定频率做某件事。CC把这个想法产品化了，而且做得更稳。

5分钟一次babysit是什么概念？你提交PR后不用再盯着，Claude会自动回复reviewer的评论、自动rebase冲突、自动跑CI修错。这不是工具了，是一个真正的AI同事在值班。

/schedule更猛。你可以设一个每天早上8点自动跑的任务：「检查所有open的issue，把P0的整理成一个列表发给我」。哪怕你电脑关了也能跑，因为它在云端执行。

4. Hooks: 在Agent生命周期中植入确定性逻辑

Boris举的例子:

- SessionStart: 每次启动Claude时动态加载上下文
- PreToolUse: 记录模型运行的每条bash命令
- PostToolUse: 自动运行lint/测试

还有个骚操作: 把权限请求路由到WhatsApp, 远程审批。

Hooks不是新功能了, 上次Boris也提过用PostToolUse做代码格式化。但这几个月Hooks系统进化了很多, 现在已经有 **25 个生命周期事件**, 从SessionStart到WorktreeCreate全覆盖, 支持正则匹配工具名, 配置文件支持全局、项目、本地三个层级。

Hooks让你能在AI做事的关键节点插入人类的规矩。AI写完代码→自动跑formatter→自动跑测试→测试不过就自动修。整个链条不需要你盯着。

把它想成CI/CD的本地版: CI是代码提交后的自动化, Hooks是代码生成时的自动化。

官方文档: <https://code.claude.com/docs/en/hooks>

5. Cowork Dispatch: 不在电脑前也能干活

Boris说他每天都用Dispatch来处理Slack和邮件、管理文件。 **不写代码的时候, 他就在Dispatch。**

Dispatch是Claude桌面应用里的一个功能, 在Cowork标签页里。你给它发一条消息描述任务, 它自己判断怎么处理:

- 如果是编码任务 (修bug、跑测试、提PR), 它自动开一个Code会话
- 如果是调研、文档、表格类工作, 它在Cowork里处理

你可以从手机发任务给Dispatch。它在你的电脑上执行, 完成后推送通知到手机。

这听着像科幻片, 但确实已经能用了。3月17日发布的Research Preview。Dispatch还支持Computer Use, Claude可以点击、滚动、操作你桌面上的应用。Max用户

(\$100/月) 先用上了, Pro用户随后开放。

6. Chrome扩展: 前端开发利器

Boris说了一句很重要的话:

使用Claude Code最重要的技巧是: **给Claude一种验证输出的方式。**

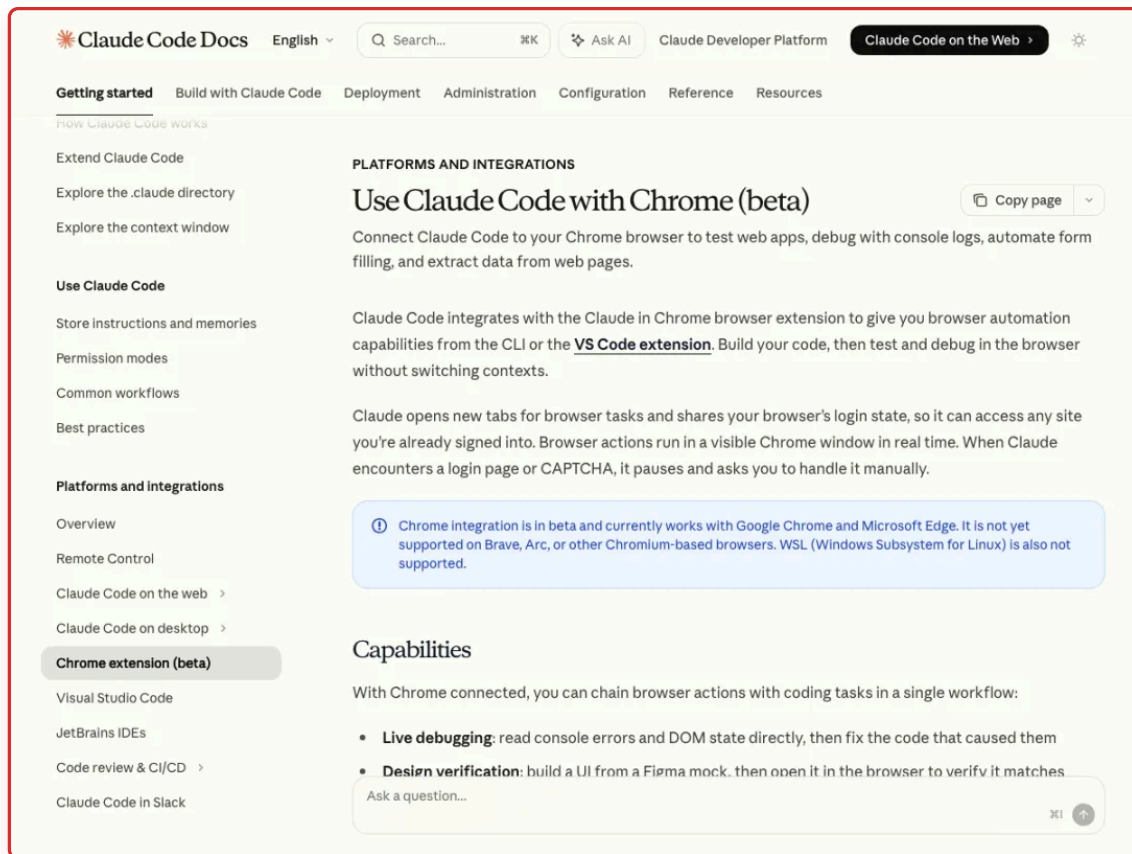
上次他在第13条说过同样的话, 这次升级成了第6条就说。可见他越来越觉得这是最核心的建议。

他的类比很到位: 如果你让一个工程师做网站, 但不让他用浏览器, 结果会好吗? 大概率不行。给Claude一个浏览器, 它就会写代码→看效果→发现问题→改代码→再看效果, 直到满意为止。

Chrome扩展的优势是: 比其他类似的MCP方案更稳定可靠。

之前我就是CC的「人类脚手架」。它开发完成, 启动本地测试服务器, 然后我去浏览器查看效果, 截图发回去, 复制控制台的报错输出粘给它。本质上我在帮CC干一些它看不到事, 当它的眼睛。

现在Chrome扩展接上了, CC完全可以自主干这些事了。自己看页面、自己截图、自己读控制台报错、自己迭代修复。我从「脚手架」变成了「验收员」, 只需要最后看一眼结果就行。



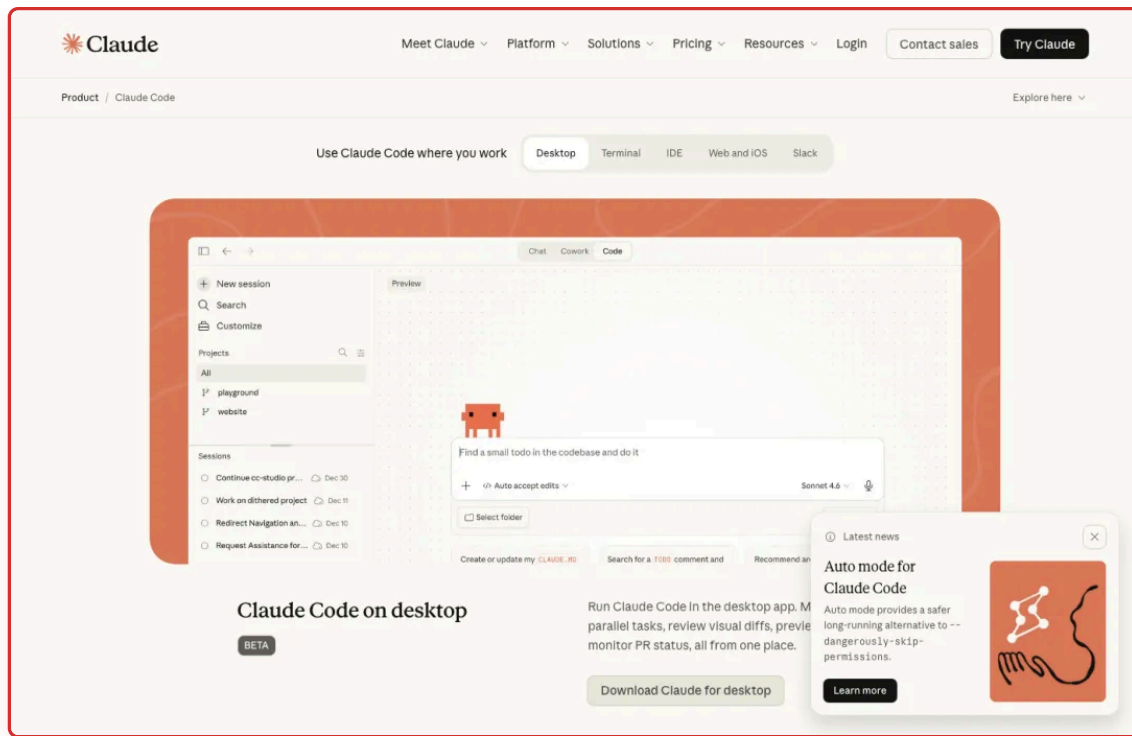
下载Chrome/Edge扩展: <https://code.claude.com/docs/en/chrome>

7. 桌面端内置浏览器预览

和Chrome扩展类似的思路，但集成度更高。

Claude桌面端可以自动启动你的dev server，然后在内置浏览器里测试。它会自动截图、检查DOM、点击元素、填表单，发现问题自己修。

这个在CLI里需要手动配置，桌面端是开箱即用。如果你主要做Web开发，桌面端的这个功能值得试试。



官方文档: <https://code.claude.com/docs/en/desktop#preview-your-app>

8. Fork会话

经常遇到这种情况: 和Claude聊了半小时, 到了一个分叉路口, 想试两个不同方向。以前只能复制粘贴开新会话, 上下文全丢了。

现在两种方式:

- 会话中运行/branch
- 从CLI运行 `claude --resume <session-id> --fork-session`

/branch会在当前位置创建一个分支会话, 两个会话共享之前的上下文, 各自往不同方向走。

这个功能看起来小, 但在实际工作中很实用。比如你在做架构设计, Claude提了两个方案, 你想两个都试试。以前得开两个新会话重新描述需求, 现在直接fork。

9. /btw: 边干活边问问题

我觉得这可能是15条里最被低估的功能。

Claude在执行一个长任务的过程中，你想问个快速问题怎么办？以前只能等它做完，或者打断它。

/btw就是在agent工作的同时做旁路查询。3月17日发布，v2.1.77版本。

› /btw 我们现在调用的是什么图片生成模型来着？

/btw 我们现在调用的是什么图片生成模型来着？

调用的是 **Google Gemini 3 Pro Image**（也叫 Nano Banana Pro），通过 Gemini API 生成和编辑图片。

具体是用 `~/.claude/skills/nano-banana-pro/scripts/generate_image.py`
Press Space, Enter, or Escape to dismissinput-image
传入垫图）。分辨率用的 2K。

它的工作原理很巧妙：Claude生成一个临时的「幽灵Agent」来回答你的问题。这个幽灵Agent是 **只读模式**，不触发工具、不修改文件，能看到当前对话的全部上下文，但 **不保存到主对话历史**，关闭后答案就消失了。它还复用父会话的Prompt缓存，额外成本极低。

Boris说他一直在用这个。想想也是，你让Claude做重构，跑了10分钟，突然想起来「对了，这个函数的返回类型是什么来着？」以前你得开个新终端查，现在直接/btw 这个函数返回什么类型？，不会污染主上下文，也不会打断Claude的工作。

10. Git Worktrees深度支持

Boris说他同时跑着几十个Claude实例，靠的就是git worktrees。

Worktree是什么？简单说，就是git允许你在同一个仓库中创建多个独立的工作目录，每个目录可以checkout不同的分支，互不干扰。

用法：`claude -w`在新的worktree中启动会话（会在`.claude/worktrees/`下创建独立目录和新分支）。也可以指定名称：`claude -w feature-auth`。2月19日发布，v2.1.49版本。Boris当时发了条帖子说：「现在Agent可以并行运行而互不干扰。」

如果你在同一个仓库里同时跑5个Claude做不同任务，它们会互相踩文件。worktree给每个Claude一个独立的沙箱，写坏了也不影响别人。

加上 `--tmux` 更香：`claude -w feature-auth --tmux` 会自动创建一个 `tmux session`，iTerm2 用户还能用原生面板。

11. `/batch`：扇出大规模变更

这个功能听着就很猛。

`/batch` 会先问你想做什么（「访谈」阶段），然后启动一个编排 Agent 进入 Plan 模式。编排 Agent 会先发射 Explore Agent 调研影响范围，再把工作拆分成 5-30 个独立执行单元，每个单元在自己的 `worktree` 里并行跑。2月28日发布，v2.1.63 版本。

Boris 说这适合大规模代码迁移和其他可并行化的工作。

举个例子：你要把一个大型项目从 CommonJS 迁移到 ESM，涉及 500 个文件。以前只能一个个改，或者让 Claude 分批处理。现在 `/batch` 自动分析哪些文件可以并行改，然后同时开几十个 agent 各干各的。

这个功能我还没实际用过，但光想想就觉得恐怖。这不是「AI 辅助编程」了，这是「AI 军团式编程」。

12. `--bare`：SDK 启动加速 10 倍

默认情况下，`claude -p` 启动时会搜索本地的 `CLAUDE.md`、`settings`、MCP 服务器、`hooks`、`skills`、`plugins`.....一大堆东西。

但很多脚本化场景不需要这些。`--bare` 跳过所有自动发现（`hooks`、`skills`、`plugins`、MCP、自动记忆、`CLAUDE.md`），只保留 Bash、文件读取和编辑三个基础工具。3月20日发布，v2.1.81 版本。

Boris 说启动速度最高提升 10 倍。这个数字取决于你装了多少 MCP、`plugins`、`skills`，装得越多，`bare` 跳过的东西越多，提速越明显。如果你本来就是裸配置，提升没那么夸张。但对写自动化脚本的人来说，哪怕每次省几百毫秒，大批量调用下来也很可观。

一个注意点：`bare` 模式下 OAuth 和 钥匙串 读取也被跳过了，你需要通过 `ANTHROPIC_API_KEY` 环境变量或 `--settings` 来认证。

用法：`claude --bare -p "你的查询"`

13. --add-dir: 跨仓库协作

跨多个仓库工作是常见场景。比如前端项目依赖一个内部UI库，两个是独立的git仓库。

`claude --add-dir ../ui-library`不仅告诉Claude另一个仓库的存在，还给了它读写权限。

Boris说他通常在一个仓库启动Claude，然后用`--add-dir`或会话中`/add-dir`加入其他仓库。2月17日起逐步完善，v2.1.45版本。

最新版本中，`--add-dir`目录下的`CLAUDE.md`和`.claude/skills/`也会被自动加载，这意味着Claude不仅能看到代码，还能理解那个仓库的规则和约定。

Claude可以同时理解两个仓库的代码，做跨仓库的修改。比如你改了UI库的接口，它能自动去前端项目里更新调用方式。

14. --agent: 自定义Agent

这是一个「经常被忽视的强大原语」，Boris原话。

做法：在`.claude/agents/`目录下定义一个agent文件，写好系统提示词和工具配置，然后`claude --agent=<你的agent名称>`启动。

用在哪？你可以创建一个专门做代码审查的agent、一个专门写文档的agent、一个专门做安全扫描的agent。每个agent有自己的性格和工具集。

这和skills不同。skills是指令集，agent是一个完整的「角色」。你甚至可以给agent配不同的MCP服务器，让它有不同的能力。

官方文档：<https://code.claude.com/docs/en/sub-agents>

15. /voice: 语音输入写代码

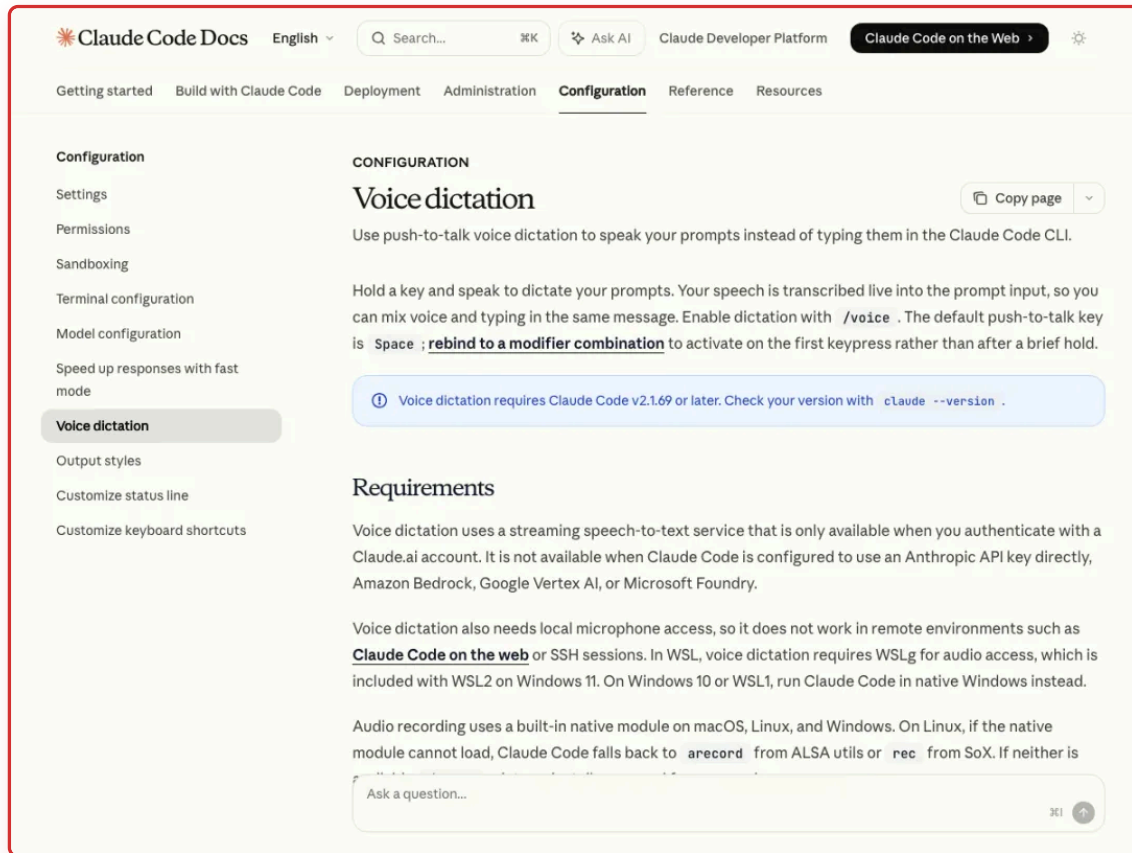
Boris说他大部分编码是 **说话** 给Claude，而不是打字。

- CLI里运行`/voice`，然后按住空格键说话，松开发送
- 桌面端点击语音按钮

- iOS设备在设置中启用听写

3月5日发布，v2.1.69版本。支持20种语言。有个好玩的细节：你可以混合打字和语音，先打一半，按住空格说后半，两种输入无缝拼接。

Claude Code的创始人，主要通过说话来写代码。



想想这意味着什么。编程的交互方式正在从「打字」变成「对话」。以前你得准确描述每一个技术细节，现在你可以像和同事聊天一样说「把那个登录页面的按钮颜色改成蓝色，然后加一个loading状态」。

我自己偶尔也用语音输入，特别是在写长prompt的时候。说比打快多了。

额外发现：Channels（频道系统）

Boris的thread里没提，但在我调研过程中发现CC新增了一个叫Channels的系统，值得一说。

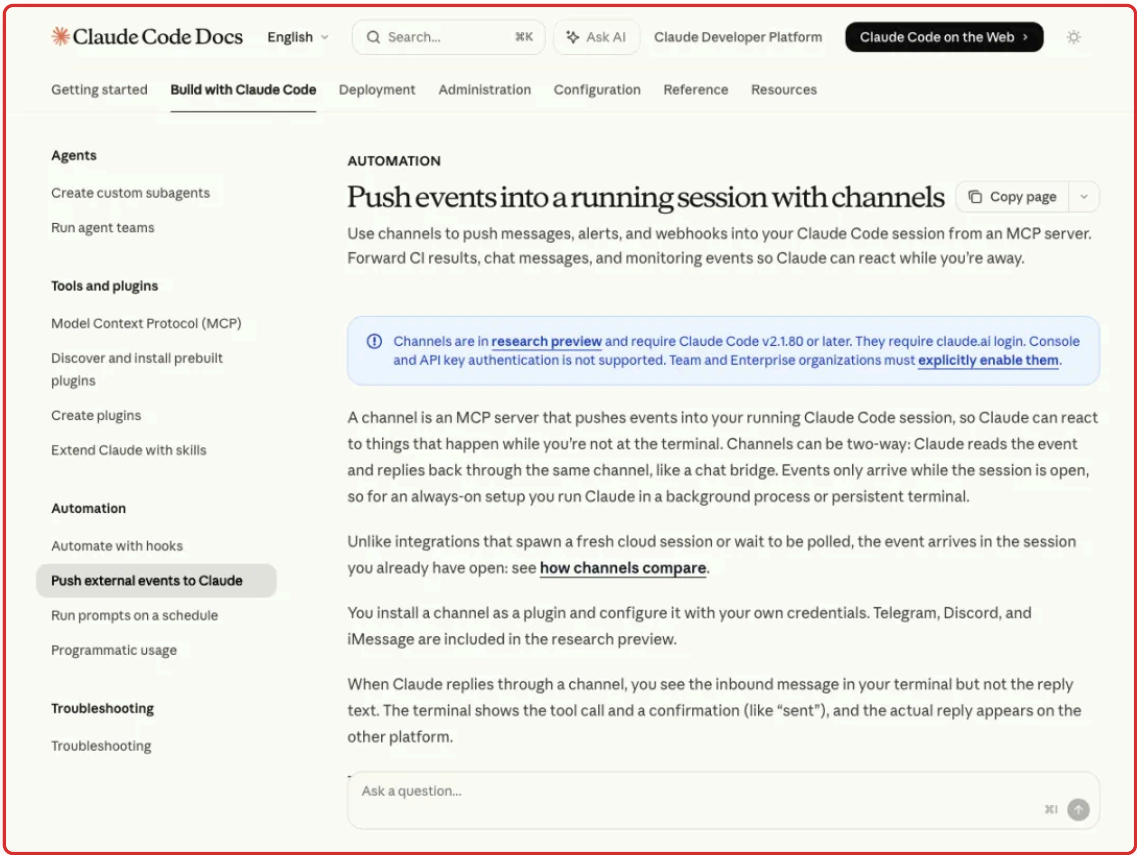
你可以通过Telegram、Discord、甚至iMessage给正在运行的Claude Code会话推送消息。不是开新会话，而是推到你已经打开的那个会话里。

比如你在终端里跑着一个CC会话做重构，出门了。你可以在Telegram上给它发消息：「顺便把那个deprecated的API也处理了」。消息直接到达正在跑的会话，Claude继续干。

设置方式：安装 Telegram 插件 → 配置 bot token → `claude --channels plugin:telegram@claude-plugins-official`启动。

这个思路和OpenClaw的消息桥接机制异曲同工。让外部消息源接入AI工作流，是Agent产品的必经之路。

目前还是Research Preview阶段，需要v2.1.80以上版本。



官方文档：<https://code.claude.com/docs/en/channels>

发布时间线

这15个功能几乎全是2026年2-3月密集发布的（v2.1.45到v2.1.81，约36个版本）：

功能	版本	日期
--add-dir 多仓库	v2.1.45	2月17日

功能	版本	日期
Git Worktrees	v2.1.49	2月19日
Remote Control	v2.1.58	2月25日
/batch 扇出执行	v2.1.63	2月28日
/voice 语音输入	v2.1.69	3月5日
/loop + /schedule	v2.1.71	3月7日
/btw + /branch	v2.1.77	3月17日
Dispatch	—	3月17日
--bare 加速启动	v2.1.81	3月20日

平均每3天一个重大功能。这个节奏说实话有点疯。

写在最后

回看这15条（加上我额外发现的Channels），和1月份的13条相比，变化非常大。

上一次，Boris的核心建议是「把基础功能用到极致」，并行跑多个Claude、好好写CLAUDE.md、用Plan模式开头。那些建议现在依然有效。

但这一次，Boris展示的是一个 **完全不同量级的产品**。移动端、跨设备、自动化循环、消息桥接、千agent并行。这些不是「技巧」，这是CC在重新定义AI编程工具的边界。

几个趋势：

编程不再绑定桌面。 手机写代码、远程控制、Dispatch从任何地方发任务。Boris自己就是这么用的，在沙发上用手机让Claude提PR。

Agent正在变得更有自主性 /loop和/schedule的本质是让AI持续运转，不需要你 在场。5分钟babysit一次、6小时升级一次依赖，这是一个永不下班的AI同事。

并行是新的生产力倍增器。 Worktrees + /batch意味着你可以同时让上千个 Claude干活。一个人的产出可以相当于一个工程团队。

OpenClaw的探索性创新正在被吸收。 移动端、消息桥接、自动化巡逻、Agent自主运转，这些概念OpenClaw先走了一步，CC在用更强的工程能力把它们做成稳定可用的产品功能。这对整个行业是好事。

我每天用Claude Code十几个小时，从写代码到写文章到做数据分析。它不完美，有时候还是会犯蠢，但这15条新功能让我觉得：CC团队确实在认真听用户的反馈，而且他们的迭代速度比任何人想象的都快。

Boris在最后说「我本来想继续写下去但不得不打住了，之后会分享更多」。

等着了。

相关链接：

- Boris的原文推文：<https://x.com/bcherny/status/2038454336355999749>
- Chrome扩展：<https://code.claude.com/docs/en/chrome>
- 桌面端预览：<https://code.claude.com/docs/en/desktop#preview-your-app>
- CLI完整参考：<https://code.claude.com/docs/en/cli-reference>
- 自定义Agent：<https://code.claude.com/docs/en/sub-agents>
- Channels频道：<https://code.claude.com/docs/en/channels>
- Remote Control：<https://code.claude.com/docs/en/remote-control>